

Labo 01 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — « Un conteneur, c'est une petite VM »

Réponse. Faux sur le point essentiel : un conteneur ne contient **pas de système d'exploitation** et n'a **pas son propre noyau**. C'est un processus de l'hôte, isolé par les *namespaces* et limité par les *cgroups*, exécuté par le noyau de l'hôte.

Pourquoi. Une VM démarre un noyau, puis un `init`, puis des dizaines de services système (journalisation, cron, SSH, réseau...) avant même que votre application ne démarre : d'où les secondes ou minutes de boot et les Go de disque. Un conteneur ne démarre rien de tout cela : le noyau tourne déjà, on lui demande juste de créer des namespaces et de lancer **un** processus. Le coût de démarrage est celui d'un `fork` + `exec`, donc quelques millisecondes ; le coût disque est celui des seules bibliothèques nécessaires à l'application.

Nuance. L'intuition « VM légère » n'est pas absurde pour un *utilisateur* : on obtient bien un `/`, un `hostname`, une IP, un `root`. Elle devient dangereuse dès qu'on parle sécurité : là où l'hyperviseur d'une VM est une vraie frontière, un conteneur partage le noyau — une faille noyau (CVE d'échappement) traverse cette frontière.

Exemple.

```
docker run --rm alpine echo bonjour    # revient en ~1 s, dont l'essentiel est le CLI
docker run -d nginx:alpine             # ~35 Mo d'image, PID visible sur l'hôte via ps
```

Question 2 — 250 Mo « sans OS »

Réponse. L'image contient l'**espace utilisateur** (*userland*) d'une distribution : `/bin/sh`, `libc`, `coreutils`, les binaires PostgreSQL, la configuration. Ce qui est **absent**, c'est le **noyau** — et avec lui tout ce qui n'existe qu'au boot : chargeur de démarrage, `initrd`, modules noyau, `systemd`, pilotes, gestion du matériel.

Pourquoi. Un programme Linux appelle le noyau par des appels système (`open`, `read`, `fork`). Il n'a pas besoin d'embarquer un noyau, il lui suffit d'en trouver un : celui de l'hôte fait l'affaire. L'image ne fournit donc que ce qui manque au-dessus.

Nuance. C'est pourquoi une image « Alpine » et une image « Debian » peuvent tourner côte à côte sur le même hôte Ubuntu : ce sont trois userlands, un seul noyau. Et c'est aussi pourquoi une image Linux ne tourne pas sur un noyau Windows.

Exemple.

```
docker run --rm alpine cat /etc/os-release    # Alpine Linux
uname -r                                     # sur l'hôte : ex. 6.8.0-generic
docker run --rm alpine uname -r              # STRICTEMENT le même noyau
```

Question 3 — Deux conteneurs nginx, deux `ps` différents

Réponse. Le **namespace** `pid`. Chaque conteneur reçoit sa propre table de PID : son processus principal y est numéroté 1 et il ne peut voir aucun PID extérieur.

Pourquoi. Voir un processus est un prérequis pour agir dessus (`kill`, `/proc/<pid>`). En retirant la visibilité, le noyau retire de fait la capacité de nuire par cette voie. Sur l'hôte, ces processus existent bien, avec de vrais PID.

Nuance. Ce n'est pas de la sécurité « au sens strict » car ce n'est **pas une frontière infranchissable** : c'est une restriction de visibilité appliquée par le même noyau que celui du conteneur. Un conteneur lancé avec `--pid=host`, `--privileged` ou avec des capacités supplémentaires retrouve la vue complète, et une faille noyau contourne le mécanisme. Le namespace isole ; il ne défend pas.

Exemple.

```
docker run -d --name web nginx:alpine
docker exec web ps -o pid,comm      # PID 1 = nginx, rien d'autre
ps -ef | grep -c nginx              # sur l'hôte : les processus sont bien là
docker run --rm --pid=host alpine ps | head # l'isolation retirée : tout l'hôte
```

Question 4 — `Cannot connect to the Docker daemon`

Réponse. Le client a fonctionné : il a affiché sa version, puis a tenté d'ouvrir la socket `/var/run/docker.sock` pour demander la version du **serveur**, et a échoué. Les deux causes probables : (1) le daemon n'est pas démarré, (2) votre utilisateur n'a pas la permission sur la socket.

Pourquoi. Toute commande Docker au-delà de `docker version --format '{{.Client...}}'` est un appel réseau vers `dockerd`. Modifier votre commande ne changera rien puisque le problème est en amont de son interprétation : personne ne l'a encore lue.

Nuance. Un troisième cas existe et trompe souvent : la variable `DOCKER_HOST` ou un *context* Docker pointe vers un daemon distant injoignable. Le message mentionne alors une autre adresse que `unix:///var/run/docker.sock` — lisez toujours l'adresse citée.

Exemple.

```
systemctl status docker      # cause 1 : inactive (dead) -> sudo systemctl start docker
ls -l /var/run/docker.sock   # srw-rw---- 1 root docker : il faut être du groupe docker
id -nG | tr ' ' '\n' | grep -x docker # cause 2 : rien -> sudo usermod -aG docker $USER
docker context ls             # cause 3 : quel daemon est ciblé ?
```

Question 5 — « Une image ne tourne pas »

Réponse. Une image est un ensemble de fichiers en lecture seule plus des métadonnées. Il n'y a aucun processus, aucun état, rien à ordonnancer : elle est aussi inerte qu'un fichier `.zip` accompagné d'une notice.

Pourquoi. Au `docker run`, Docker ajoute trois choses : (1) une **couche d'écriture** fine par-dessus les couches en lecture seule, (2) un jeu de **namespaces et de cgroups**, (3) l'**exécution** de la commande inscrite dans les métadonnées de l'image (`ENTRYPOINT` / `CMD`). Le résultat de cet assemblage est le conteneur.

Nuance. Le conteneur *créé* et le conteneur *démarré* sont deux étapes distinctes : `docker create` fait tout sauf lancer le processus, `docker start` le lance. `docker run` est simplement `create` + `start` (+ `pull` si l'image est absente).

Exemple.

```
docker create --name tmp alpine sleep 30 # conteneur créé, aucun processus
docker ps -a --filter name=tmp          # STATUS : Created
docker start tmp && docker ps            # STATUS : Up -> maintenant il tourne
docker rm -f tmp
```

Question 6 — Données PostgreSQL après un `docker rm`

Réponse. Non, les données ont disparu. Et non, l'image n'a **pas** été modifiée : elle est strictement identique avant et après.

Pourquoi. Les écritures d'un conteneur (via *copy-on-write*) atterrissent dans sa couche d'écriture privée, propre à cette instance. `docker rm` supprime le conteneur **et** cette couche. Les couches de l'image sont en lecture seule : rien de ce que fait un conteneur ne peut les altérer — c'est ce qui garantit que deux conteneurs de la même image partent du même état.

Nuance. Deux corrections importantes. D'abord, `docker stop` ne détruit rien : un conteneur arrêté conserve sa couche, et `docker start` retrouve les données. C'est bien `rm` qui détruit. Ensuite, l'image officielle `postgres` déclare `/var/lib/postgresql/data` comme `VOLUME` : Docker crée alors un volume **anonyme** qui, lui, survit au `rm` — mais sans nom, il est impossible à retrouver et sera balayé au premier `docker volume prune`. En pratique, on considère les données perdues. La solution propre — le volume nommé — est l'objet du labo 06.

Exemple.

```
docker run -d --name pg -e POSTGRES_PASSWORD=x postgres:16-alpine
docker exec pg psql -U postgres -c "CREATE TABLE t(id int);"
docker rm -f pg
docker run -d --name pg -e POSTGRES_PASSWORD=x postgres:16-alpine
docker exec pg psql -U postgres -c "\dt" # Did not find any relations.
```

Question 7 — Dix conteneurs d'une image de 280 Mo

Réponse. Quelques kilo-octets au total, pas 2,8 Go. Les 280 Mo ne sont stockés **qu'une fois**.

Pourquoi. Les couches de l'image sont partagées en lecture seule entre toutes les instances. Chaque conteneur ne possède en propre que sa couche d'écriture, initialement vide. Le pilote de stockage (`overlay2`) empile les couches et n'y copie un fichier que si le conteneur le modifie : c'est le *copy-on-write*.

Nuance. « Initialement vide » est le point de vigilance : un conteneur qui écrit des logs applicatifs ou des fichiers temporaires dans son système de fichiers fait grossir sa couche indéfiniment, et cela ne se voit ni dans `docker images` ni dans `df` de façon évidente. `docker ps -s` révèle cette taille (colonne `SIZE`, partie hors `virtual`).

Exemple.

```
for i in $(seq 1 10); do docker run -d --name lab$i alpine sleep 300; done
docker ps -s --format '{{.Names}} {{.Size}}' # ~4.1kB (virtual 9.1MB) partout
docker rm -f $(docker ps -aq --filter name=lab)
```

Question 8 — Bind mount et daemon distant

Réponse. Le chemin `/home/dev/data` a été résolu sur le **serveur distant** où tourne `dockerd`, pas sur le poste du développeur. Ce dossier n'y existant probablement pas, Docker l'a créé vide, puis l'a monté vide dans le conteneur.

Pourquoi. Le client `docker` ne transmet que du texte : il envoie la chaîne `/home/dev/data` au daemon dans la requête HTTP. C'est le daemon qui ouvre le chemin, sur son propre système de fichiers. Le client n'envoie jamais le contenu d'un dossier lors d'un `run`.

Nuance. Le `docker build` fonctionne à l'inverse et c'est ce qui rend le sujet confus : là, le client **empaquette** le dossier courant (le *build context*) dans une archive et l'envoie au daemon. D'où le message `Sending build context to Docker daemon 2.5 MB`. Règle simple : *build* → contenu transféré ; *bind mount* → simple chaîne de caractères interprétée à distance.

Exemple.

```
docker context ls # vérifier quel daemon on pilote
echo $DOCKER_HOST # ssh://user@serveur ou tcp://...
docker run --rm -v /home/dev/data:/data alpine ls -la /data # vide : preuve
```

Question 9 — Pourquoi le groupe `docker` est sensible

Réponse. Parce que l'appartenance au groupe `docker` équivaut à un accès **root** sur l'hôte, sans passer par `sudo` et donc sans laisser de trace exploitable.

Pourquoi. Le daemon tourne en root. Qui peut lui parler peut lui demander de monter n'importe quel chemin de l'hôte dans un conteneur — y compris `/`. Une seule commande suffit alors à lire `/etc/shadow` ou à modifier `/etc/sudoers` de l'hôte, sans aucune faille, en utilisant Docker exactement comme prévu.

Nuance. `sudo docker ...` n'est pas plus « sûr » techniquement : le pouvoir final est le même. Ce que l'entreprise gagne est ailleurs : la traçabilité (journal `sudo`), la possibilité de restreindre nominativement, et le fait que l'élévation devient un geste conscient. La vraie réduction de risque technique passe par le *rootless mode* ou par l'interdiction d'accès direct au daemon en production.

Exemple.

```
# Démonstration (à ne PAS faire sur un serveur partagé) :
docker run --rm -v /:/hote alpine cat /hote/etc/shadow
# Aucune faille : c'est le fonctionnement normal du produit.
```

Question 10 — Formes longues

Réponse.

Raccourci	Forme longue	Objet
<code>docker ps -a</code>	<code>docker container ls -a</code>	conteneurs
<code>docker images</code>	<code>docker image ls</code>	images
<code>docker rmi nginx:alpine</code>	<code>docker image rm nginx:alpine</code>	image
<code>docker rm web</code>	<code>docker container rm web</code>	conteneur

Pourquoi. La grammaire `docker <objet> <action>` a été introduite en 2017 (Docker 1.13) pour mettre de l'ordre dans une CLI devenue foisonnante. Les anciennes commandes ont été conservées telles quelles pour ne casser ni les scripts ni les habitudes.

Nuance. `ps` et `images` viennent d'univers différents : `ps` est emprunté à UNIX (« lister les processus ») et désigne donc les conteneurs, tandis que `images` est un simple pluriel. D'où l'asymétrie déroutante : `docker ps` liste des conteneurs, `docker rm` supprime un conteneur, mais `docker rmi` supprime une image — le `i` final est la seule différence entre deux commandes destructrices.

Exemple.

```
docker container ls -a --format 'table {{.Names}}\t{{.Status}}'
docker image ls --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
```

Question 11 — Un noyau Linux sur macOS

Réponse. Il voit le noyau d'une **machine virtuelle Linux** que Docker Desktop fait tourner en arrière-plan. Le daemon et tous les conteneurs vivent dans cette VM ; le binaire `docker` du Mac n'est qu'un client qui lui parle.

Pourquoi. Un conteneur Linux exige des namespaces et des cgroups Linux. macOS n'en a pas. La seule façon d'exécuter des conteneurs Linux est donc de fournir un noyau Linux — via une VM (LinuxKit sous Docker Desktop, ou Apple Virtualization).

Nuance. Conséquence à connaître : sur macOS et Windows, l'argument « pas de VM, donc léger » tombe. Vous payez bien une VM (RAM réservée, disque, démarrage), simplement une seule pour tous vos conteneurs au lieu d'une par application. Les entrées/sorties sur fichiers montés depuis l'hôte y sont notablement plus lentes qu'en natif — un grief classique des développeurs Java et Node. Sur les serveurs Linux de production, en revanche, l'affirmation reste vraie.

Exemple.

```
docker run --rm alpine uname -a
# Sur Linux : le noyau de votre poste.
# Sur macOS : ex. "linuxkit" — le noyau de la VM cachée.
```

Question 12 — Conteneur qui dévore la RAM

Réponse. Non, le namespace `pid` n'y peut rien : il cache les processus, il ne limite aucune consommation. Le mécanisme correct est le **cgroup**, exposé par les options `--memory` et `--cpus`. Sans limite configurée, le conteneur peut consommer toute la mémoire de l'hôte.

Pourquoi. Namespaces et cgroups répondent à deux questions différentes : « que puis-je voir ? » et « combien puis-je consommer ? ». Un conteneur sans `--memory` hérite de la totalité de la RAM

disponible ; c'est alors l'*OOM killer* du noyau qui arbitre, à l'échelle de l'hôte, et il peut très bien tuer un **autre** processus que le fautif.

Nuance. Pour la JVM, il y a un piège supplémentaire. Les JDK modernes (11+) détectent le cgroup et dimensionnent le tas en conséquence (`MaxRAMPercentage` , 25 % par défaut). Poser `--memory=512m` sans y penser peut donc réduire silencieusement le tas de votre Spring Boot à ~128 Mo et provoquer des `OutOfMemoryError` qui n'existaient pas hors conteneur. Limiter est nécessaire, mais jamais « gratuit ».

Exemple.

```
docker run -d --name gourmand --memory=256m --memory-swap=256m alpine sh -c \
    'while true; do ;; done'
docker stats --no-stream gourmand      # MEM USAGE / LIMIT : ... / 256MiB
docker rm -f gourmand
```

Question 13 — Promouvoir une image sans la reconstruire

Réponse. L'**immutabilité** de l'image, et son identification par un **digest** (empreinte cryptographique de son contenu). Le binaire testé en intégration est bit à bit celui déployé en production.

Pourquoi. Un digest (`sha256:...`) est calculé sur le contenu : deux images de même digest sont le même artefact. On peut donc tester une fois et déployer le même objet partout, ce qui supprime toute une classe d'incidents « ça passait en recette ».

Nuance. Reconstruire à chaque étape « à partir du même code source » ne donne **pas** le même résultat : entre deux builds, l'image de base a pu être mise à jour (`21-jre` pointe vers un nouveau patch), un `apt-get install` a tiré une version plus récente, une dépendance non figée a bougé. On teste alors une image et on en déploie une autre. C'est aussi pourquoi, en entreprise, on déploie par digest ou par tag immuable (`api:1.4.2-build.318`) plutôt que par `latest` .

Exemple.

```
docker image inspect --format '{{index .RepoDigests 0}}' postgres:16-alpine
# postgres@sha256:... <- l'identité stable, indépendante du tag
```

Question 14 — Client ou daemon ?

Réponse et justification.

Opération	Où	Pourquoi
(a) Lecture du <code>Dockerfile</code>	Client	Il l'ajoute au <i>build context</i> et transmet le tout au daemon, qui exécute les instructions
(b) Affichage du tableau <code>docker ps</code>	Client	Le daemon renvoie du JSON ; le formatage, les colonnes et les couleurs sont côté client
(c) <code>docker pull</code> depuis Docker Hub	Daemon	C'est lui qui parle au registry et stocke les couches ; le client ne fait qu'afficher la progression
(d) Résolution du <code>.</code> dans <code>docker build .</code>	Client	Le <code>.</code> désigne le dossier courant du client , qui l'empaquette et l'envoie

Nuance. Le point (c) a une conséquence directe en entreprise : les identifiants de registry (`docker login`) sont stockés côté client (`~/.docker/config.json`) mais c'est le daemon qui télécharge — et c'est donc le **daemon** qui doit être capable d'atteindre le registry à travers le proxy d'entreprise. Configurer le proxy dans votre shell ne suffit pas ; il faut le configurer pour le service `dockerd`.

Exemple.

```
# Le proxy du daemon, pas celui de votre terminal :  
systemctl show docker --property=Environment  
docker info | grep -i proxy
```